Advance Python Hand-Book

1. pip, Modules, Types of Modules, and Packages in Python

Definition : pip is the default package manager for Python used to install, upgrade, and manage external libraries and dependencies from the Python Package Index (PyPI) or other repositories.

Explanation: pip allows developers to extend Python's functionality by installing third-party packages. It simplifies dependency management and ensures that projects use the correct versions of libraries.

pip works with:

- Global Python environments
- Virtual environments
- Requirement files for project dependency tracking

Common pip Commands:

Command	Purpose
pip install package_name	**Install a package**
pip uninstall package_name	**Remove a package**
pip list	**Show installed packages**
pip show package_name	**Display package details**
pip freeze	**Output installed packages with versions**
pip install -r requirements.txt	**Install from requirement file**
pip install --upgrade package_name	**Upgrade a package**

Key Features of pip

- Dependency resolution
- Version management
- Installation from local or remote sources
- Integration with virtual environments

Modules in Python

Definition : A module is a single Python file containing reusable code such as functions, classes, and variables that can be imported into other programs.

Explanation
Modules promote:

- Code reusability
- Maintainability
- Logical organization
- Namespace separation

Python automatically treats any .py file as a module.
Advantages of Modules

Dhruv Patel 120

Advantage	Description
Reusability	**Code can be reused across projects**
Organization	**Separates logic into manageable parts**
Maintainability	**Easier debugging and updates**
Namespace control	**Avoids naming conflicts**

Module search path

Python searches for modules in this order:

1. Current directory
2. Standard library directories
3. Installed packages (site-packages)

You can check it using:

```
import sys
print(sys.path)
```

__name__ == "__main__" concept

Very common interview question.

```
if __name__ == "__main__":
    print("Running directly")
```

Facts:

- When a file runs directly → __name__ is "__main__"
- When imported → __name__ is module name
- Used to separate testing code from reusable code

Types of Modules

Python modules are categorized based on their origin and purpose.

1. Built-in Modules

Definition: Modules that come pre-installed with Python.

Examples: math, os, sys, datetime

Characteristics:

- No installation required
- Optimized and tested
- Platform-independent

2. User-Defined Modules

Definition: Modules created by developers to organize their own code.

Characteristics:

- Custom functionality
- Project-specific logic

DhruvPatel16120

- Easy to modify and extend

3. Third-Party Modules
Definition: Modules developed by the community and installed using pip.
Examples: numpy, pandas, flask, requests
Characteristics:
- Installed via pip
- Open-source ecosystem
- Extend Python capabilities

Some important Questions :

Q1. What is if __name__ == "__main__": main() in Python?
It is a standard Python pattern that ensures the main() function runs **only when the file is executed directly**, not when it is imported as a module.

Q2. What is __name__ in Python?
__name__ is a built-in special variable that stores the name of the current module.
- If the file runs directly → __name__ is "__main__"
- If imported → __name__ is the module's filename

Q3. Why do we use if __name__ == "__main__"?
It prevents certain code (usually testing or execution code) from running when the file is imported into another program.

It helps separate:
- reusable functions
- executable program logic

Q4. What is the purpose of calling main() inside it?
Calling main() organizes the program by placing the main logic inside a function instead of running code globally. This improves readability and structure.

Example:

```
def main():
    print("Program is running")
if __name__ == "__main__":
    main()
```

Q5. What happens if we remove this condition?
All top-level code runs automatically when the file is imported, which may cause unwanted execution.

Q7. What are the benefits of this pattern?

- Prevents accidental execution on import
- Supports modular programming
- Allows easy testing
- Improves code organization

DhruvPatel1610

Q8. Short interview definition

if __name__ == "__main__": main() is used to execute the main function only when a Python file runs directly, ensuring that the code does not run automatically when imported as a module.

Comparison of Module Types

Type	Source	Installation Required	Example
Built-in	Python standard library	No	os, math
User-defined	Created by user	No	Custom .py files
Third-party	External developers	Yes (via pip)	numpy, flask

Packages in Python

What is a Package in Python?

Definition: A **Python package** is a directory that contains multiple related modules organized in a hierarchical structure to build large and maintainable applications. A package allows developers to group logically related modules together and manage namespaces effectively.

Why Packages are Important

Packages solve problems that appear in large programs:

Problem Without Packages	How Packages Help
Code becomes messy	Organized folder structure
Name conflicts occur	Namespace separation
Hard to maintain large codebases	Modular architecture
Difficult collaboration	Clear project structure

Basic Structure of a Python Package

A standard Python package has the following structure:

```
my_package/
  │
  ├── __init__.py
  ├── module1.py
  ├── module2.py
  └── subpackage/
        ├── __init__.py
        └── module3.py
```

Component	Purpose
Package folder	Main container of modules
__init__.py	Marks directory as a package
Modules (.py files)	Contain program logic
Subpackages	Nested packages for hierarchy

Dhruv Patel 1612

Role of __init__.py

Definition : __init__.py is a special file executed when a package is imported.

Functions of __init__.py

- Initializes package variables
- Controls what is exported when importing
- Can execute setup code
- Makes the folder recognizable as a package

Types of Packages:

Type	Description
Regular Package	Contains __init__.py
Namespace Package	No __init__.py (Python 3.3+)

How to Create a Python Package (Step-by-Step)

Step 1: Create a Folder
Create a main directory with the package name.
mkdir my_package

Step 2: Add __init__.py
Inside the folder, create an empty initialization file.
touch __init__.py

Step 3: Add Modules
Create Python files inside the package.
module1.py
module2.py

Step 4: Create Subpackages (Optional)
You can create nested folders with their own __init__.py.

Step 5: Import the Package
Use import statements to access modules.

Import styles:

Import Style	Example
Import whole module	import my_package.module1
Import specific item	from my_package.module1 import function
Alias import	import my_package.module1 as m1

DhruvPatel16120

Absolute vs Relative Imports

Absolute Import
Uses full path from the project root.
from my_package.module1 import function

Relative Import
Uses dot notation inside packages.
from .module1 import function
from ..subpackage.module3 import function

Comparison

Feature	Absolute Import	Relative Import
Readability	Clear and explicit	Shorter
Portability	Better	Limited to package
Usage	Recommended for large projects	Internal package use

Package Distribution and Installation

Packages can be shared and installed using pip.
Creating an Installable Package
Key files required:

File	Purpose
setup.py or pyproject.toml	Package metadata
README.md	Documentation
requirements.txt	Dependencies
LICENSE	Legal permissions

Types of Python Packages

1. Standard Library Packages
Built into Python.
Examples: os, json, collections

2. Third-Party Packages
Installed via pip.
Examples: numpy, django, requests

3. Custom Packages
Created by developers for specific projects.

Best Practices for Package Design

Dhruv Patel 1b120

Practice	Explanation
Use meaningful names	Improves readability
Keep modules focused	One responsibility per module
Avoid circular imports	Prevent dependency issues
Use documentation	Helps maintainability
Follow PEP 8	Standard Python style

Package vs Module vs Library

Feature	Module	Package	Library
Structure	Single file	Folder of modules	Collection of packages
Scope	Small	Medium	Large
Example	math.py	my_package/	NumPy

Advantages of Using Packages

- Scalable project architecture
- Reusable components
- Clean namespace management
- Easier collaboration
- Simplified deployment

Video for modules and packages: https://youtu.be/4ubEUFgnbLI?si=KEa7Ta_1WqULuIja

File Handling in Python

What is File Handling?
Definition: File handling in Python is the process of creating, opening, reading, writing, and managing files stored on a computer system using Python programs.

It allows programs to store data permanently instead of keeping it only in temporary memory (RAM).

Purpose of File Handling

File handling is essential for working with real-world applications where data must be saved, retrieved, and updated.

DhruvPatel1120

Common Uses of File Handling

Use Case	Description
Data storage	Save program output permanently
Data retrieval	Load stored information for processing
Logging	Record system or application events
Configuration	Store application settings
Data exchange	Share data between programs

Types of Files in Python

Python mainly works with two categories of files:

1. Text Files
Text files store data in human-readable form using characters.
Examples: .txt, .csv, .json, .xml
Characteristics:

- Readable by humans
- Stored as plain text
- Easy to edit

2. Binary Files

Binary files store data in machine-readable format.
Examples: .bin, .jpg, .mp3, .exe
Characteristics:

- Not human-readable
- Compact storage
- Faster processing for complex data

Comparison of File Types

Feature	Text Files	Binary Files
Readability	Human-readable	Machine-readable
Storage format	Characters	Binary data
Editing	Easy with text editors	Requires special tools
Speed	Slower for large data	Faster and efficient

File Handling Workflow in Python

File handling typically follows a structured sequence:

1. Open the file
2. Perform operations (read/write)

DhruvPatel16120

3. Close the file

This ensures safe resource management and prevents data corruption.

File Handling Modes Overview

Files are accessed using different modes depending on the operation required.

Mode	Purpose
Read mode	Open file to read data
Write mode	Create or overwrite file
Append mode	Add data to existing file
Binary mode	Handle non-text data

Detailed discussion of these modes appears in the next section on file operations.

Importance of File Handling in Programming

File handling enables:

- Persistent data storage
- Efficient data management
- Automation of repetitive tasks
- Large-scale data processing

It is a foundational concept in software development, data science, and system programming.

Key Concepts in File Handling

Concept	Explanation
File object	Interface used to interact with files
File path	Location of a file in the system
Access mode	Determines how the file is used
Buffering	Temporary memory for efficient I/O

Text File Operations in Python (.txt, .csv, .json, .xml)

Overview of Text File Operations

Text file operations allow Python programs to store, retrieve, and manipulate structured and unstructured data. Python provides built-in functions and libraries to handle different text formats efficiently.

General File Operation Workflow

Step	Description
Open	Access file using appropriate mode
Read/Write	Perform operations on file content

Dhruv Patel 16120

Process	Modify or analyze data
Close	Release system resources

File Opening Modes (Common to All Text Files)

Mode	Meaning
r	Read (default)
w	Write (overwrite)
a	Append
r+	Read and write
w+	Write and read
a+	Append and read

Using with Statement (Recommended)

Definition: The **with statement** automatically manages file opening and closing. It ensures the file is safely closed even if errors occur.

Advantages

- Prevents resource leaks
- Cleaner syntax
- Automatic exception handling

1. Text File Operations (.txt)

Reading Text Files
Common Reading Methods

Method	Description
read()	Reads entire file
readline()	Reads one line
readlines()	Reads all lines into list

Writing Text Files
Writing operations create or modify text content.

Key concepts:

- Overwriting vs appending
- Line formatting
- Encoding handling

Functions for file writing

DhruvPatel16120

Function	Purpose
open()	Opens file
close()	Closes file
write()	Writes text
writelines()	Writes multiple lines

File pointer operations

tell(): Returns current cursor position.
pos = f.tell()

seek(): Moves cursor to position.
f.seek(0) # go to start

2. CSV File Operations (.csv)

What is a CSV File?
CSV (Comma-Separated Values) files store tabular data where values are separated by commas. Python uses the built-in **csv module** for handling CSV files.

Reading CSV Files

Key tools:
- CSV reader objects
- Iterating through rows
- Handling headers

Writing CSV Files
Python provides structured writing tools:

Feature	Purpose
CSV writer	Writes rows to file
Delimiters	Custom separators
Quoting	Handles special characters

Advanced CSV Concepts
- Dictionary-based reading/writing
- Custom delimiters
- Large dataset streaming

3. JSON File Operations (.json)

DhruvPatel16120

What is JSON?

JSON (JavaScript Object Notation) is a lightweight data-exchange format used for APIs and configuration files.
Python handles JSON using the **json module**.

JSON Data Types Mapping:

JSON Type	Python Equivalent
Object	Dictionary
Array	List
String	String
Number	int/float
Boolean	True/False
Null	None

Reading JSON Files

Processes include:

- Parsing JSON into Python objects
- Error handling for invalid data

Writing JSON Files

Important features:

- Serialization
- Formatting and indentation
- Pretty printing

Advanced JSON Features

- Custom encoders/decoders
- Handling nested structures
- Streaming large JSON files

4. XML File Operations (.xml)

What is XML?

XML (eXtensible Markup Language) stores structured hierarchical data using tags.
Python supports XML handling through **xml.etree.ElementTree** and other libraries.

XML Structure Components

Component	Description
Elements	Main building blocks

DhruvPatel16120

Tags	Define structure
Attributes	Additional information
Tree structure	Parent-child hierarchy

Reading XML Files

Key operations:

- Parsing XML tree
- Navigating elements
- Extracting values

Writing XML Files

Processes include:

- Creating elements
- Modifying trees
- Saving structured data

Advanced XML Topics

- XPath queries
- Namespaces
- XML validation

Error Handling in File Operations

File operations may raise exceptions.

Common Errors

Error	Cause
FileNotFoundError	File does not exist
PermissionError	Access denied
IOError	Input/output failure
JSONDecodeError	Invalid JSON

Using exception handling improves reliability.

Best Practices for Text File Handling

Practice	Reason
Use with statement	Safe file management
Validate file paths	Prevent runtime errors

DhruvPatel16120

Handle exceptions	Improve robustness
Use proper encoding	Avoid character issues
Close unused files	Free system resources

Performance Considerations

Advanced programs should consider:

- Buffered reading/writing
- Streaming large files
- Memory efficiency
- Lazy loading techniques

Introduction to Binary Files

Definition: A **binary file** stores data in raw byte format rather than human-readable text. The data is encoded in binary (0s and 1s) and is interpreted by programs instead of text editors.
Binary files are used for images, audio, videos, compiled programs, and serialized objects.

Characteristics of Binary Files

Feature	Description
Format	Stored as raw bytes
Readability	Not human-readable
Speed	Faster for large data
Precision	No encoding conversion loss
Usage	Multimedia and structured data

Binary File Modes in Python

Binary files are opened by adding **b** to file modes.

Mode	Purpose
rb	Read binary file
wb	Write binary file
ab	Append binary data
rb+	Read and write binary
wb+	Write and read binary

Using the with Statement for Binary Files
The with statement is recommended for binary operations because it automatically manages file closing and error handling.

bhrutipatel1610120

Binary operations follow the same workflow:

1. Open file in binary mode
2. Read or write byte data
3. Automatically close file

Reading Binary Files

Binary files are read as **bytes objects**.

Key Reading Methods

Method	Description
read()	Reads entire binary file
read(n)	Reads specific number of bytes
seek()	Moves file pointer
tell()	Returns current position

File Pointer Control

Binary files use a pointer to track reading/writing position.

Function	Purpose
seek(offset)	Move pointer location
tell()	Get pointer position

Writing Binary Files

Binary writing stores raw byte data directly to disk.

Writing Methods

Method	Description
write(bytes)	Writes byte data
flush()	Forces write to disk

Binary writing requires **bytes-like objects**, not strings.

Working with Byte Data

Bytes vs Strings

Feature	Bytes	Strings
Data type	bytes	str
Encoding	Raw binary	Text encoding
Prefix	b''	''

DhruvPatel16120

Usage	Binary files	Text files

Conversion between text and binary requires encoding and decoding.

Serialization with Binary Files

Binary files are often used to store Python objects using serialization.

Pickle Module Overview

The **pickle** module converts Python objects into binary format and restores them later.

Operation	Purpose
Serialization	Convert object to bytes
Deserialization	Restore object

Use Cases

- Saving program state
- Machine learning models
- Caching complex objects

Handling Large Binary Files

Advanced programs use efficient strategies:

Technique	Benefit
Chunk reading	Reduces memory usage
Streaming	Processes data gradually
Buffering	Improves performance

Error Handling in Binary Operations

Common exceptions:

Error	Cause
FileNotFoundError	Missing file
PermissionError	Access denied
EOFError	Unexpected end of file
IOError	Input/output issues

Proper exception handling ensures safe execution.

Best Practices for Binary File Handling

Practice	Reason
Always use binary mode (b)	Prevent encoding errors
Use with statement	Automatic resource management

bhuvpatel16120

Read in chunks for large files	Memory efficiency
Validate file paths	Avoid runtime errors
Handle exceptions	Improve reliability

Binary vs Text File Operations

Feature	Binary Files	Text Files
Data format	Raw bytes	Human-readable text
Speed	Faster	Slower
Encoding issues	None	Possible
Use cases	Media, serialized data	Documents, logs

Summary

Binary file handling in Python:

- Works with raw byte data
- Supports pointer control and chunk processing
- Enables object serialization with pickle

Binary operations are essential for handling multimedia files, system-level programming, and efficient data storage.Top of Form

Exception Handling in Python

Definition : Exception handling in Python is a mechanism for detecting and managing runtime errors so that a program can continue executing in a controlled manner instead of crashing.
An **exception** is an event that interrupts the normal flow of a program.

Why Exception Handling is Important
Without exception handling, programs terminate when errors occur. Exception handling improves reliability and robustness.
Benefits

Benefit	Explanation
Program stability	Prevents sudden crashes
Error control	Handles unexpected situations
User experience	Provides meaningful messages
Debugging support	Identifies error sources
Resource safety	Ensures cleanup operations

Types of Errors in Python

Python errors are broadly classified into:
1. Syntax Errors

DhruvPatel16120

Errors caused by incorrect code structure.
Example causes:

- Missing parentheses
- Incorrect indentation
- Invalid keywords

These are detected before execution.

2. Runtime Errors (Exceptions)
Errors that occur during program execution.
Examples:

- Dividing by zero
- Accessing invalid index
- Opening non-existent file

These are handled using exception handling.

Basic Structure of Exception Handling

Python uses structured blocks to manage exceptions.
Core Components

Keyword	Purpose
try	Contains risky code
except	Handles exceptions
else	Executes if no exception occurs
finally	Always executes (cleanup)

Execution Flow

1. Code inside try runs
2. If an error occurs → except executes
3. If no error → else executes
4. finally runs in all cases

2. Errors vs Exceptions

Error: An **error** is a problem that prevents the program from running correctly.

Two main categories:
1 Syntax Errors
2 Runtime Errors (Exceptions)

Exception: An **exception** is an object that represents an error during program execution.

When an exception occurs:

1. Python creates an exception object
2. Normal execution stops
3. Control transfers to exception handling code

4. Types of Exceptions in Python
Common built-in exceptions

DhruvPatel16120

Exception	Description
ZeroDivisionError	Division by zero
TypeError	Wrong data type operation
ValueError	Invalid value
IndexError	Invalid list index
KeyError	Missing dictionary key
FileNotFoundError	File does not exist
NameError	Undefined variable
AttributeError	Invalid attribute
ImportError	Import failure
OverflowError	Numeric overflow

All exceptions inherit from:

```
BaseException
    └── Exception
```

Raising Exceptions Manually

Definition: Python allows programmers to generate exceptions intentionally using the **raise** statement.

Purpose

- Enforce rules
- Validate input
- Stop invalid execution

5. try–except block

Definition: Used to catch and handle exceptions.

Syntax

```
try:
    # risky code

except ExceptionType:
    # handling code
```

Example

```
try:
    x = 10 / 0

except ZeroDivisionError:
    print("Cannot divide by zero")
```

Multiple except blocks

DhruvPatel1b120

```
try:
    x = int("abc")

except ValueError:
    print("Invalid number")

except TypeError:
    print("Type error")
```

Catching multiple exceptions together

```
except (ValueError, TypeError):
    print("Error occurred")
```

6. try–except–else block

Definition: The else block runs only if no exception occurs.

Syntax

```
try:
    code

except:
    handling

else:
    success code
```

Example

```
try:
    x = int("10")

except ValueError:
    print("Conversion failed")

else:
    print("Conversion successful")
```

7. try–except–finally block

Definition: The finally block always executes, whether an exception occurs or not. Used for cleanup operations.

Syntax

```
try:
    code

except:
    handling

finally:
    cleanup
```

Example

```
try:
```

DhruvPatel1612O

```
    f = open("data.txt")

except FileNotFoundError:
    print("File missing")

finally:
    print("Execution finished")
```

8. raise statement (throwing exceptions)
Definition: The raise keyword is used to manually trigger an exception.
Syntax

```
raise ExceptionType("message")
```

Example

```
age = -1
if age < 0:
    raise ValueError("Age cannot be negative")
```

This is often called **throwing an exception**.

9. Re-raising exceptions
You can re-throw a caught exception:

```
try:
    x = 1 / 0
except ZeroDivisionError:
    print("Error occurred")
    raise
```

10. Custom Exception Classes

Why create custom exceptions?

- For application-specific errors
- Better debugging
- Cleaner error handling

Creating a custom exception
Custom exceptions inherit from Exception.

```
class InvalidAgeError(Exception):
    pass
```

Using custom exception

```
age = -5
if age < 0:
    raise InvalidAgeError("Invalid age value")
```

Custom exception with constructor

DhruvPatel16120

```
class BalanceError(Exception):
    def __init__(self, message):
        self.message = message
        super().__init__(self.message)
```

11. Exception hierarchy (simplified)

```
BaseException
    ├── SystemExit
    ├── KeyboardInterrupt
    └── Exception
            ├── ArithmeticError
            ├── LookupError
            ├── ValueError
            ├── TypeError
            └── FileNotFoundError
```

12. Best practices in exception handling

- Catch specific exceptions, not generic ones
- Avoid empty except blocks
- Use finally for cleanup
- Use custom exceptions for clarity
- Do not overuse exception handling

13. Advantages of exception handling

- Prevents program crashes
- Improves reliability
- Separates error-handling logic
- Makes debugging easier

14. Common interview questions

Q: What is exception handling?
A: A method to handle runtime errors gracefully.

Q: Difference between error and exception?
A: Errors are syntax issues; exceptions occur during execution.

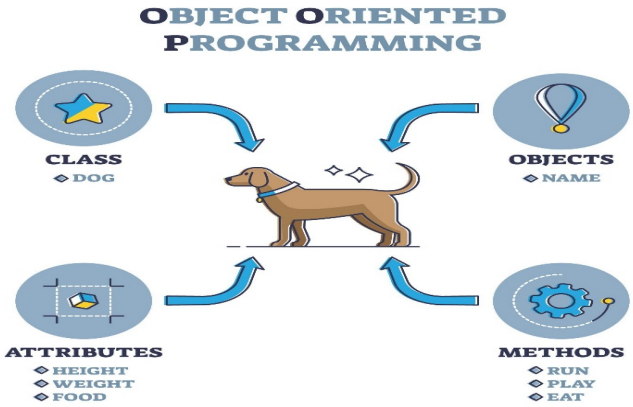
Q: Purpose of finally block?
A: Executes cleanup code always.

Q: What is raise?
A: Used to manually throw exceptions.

Q: How to create custom exceptions?
A: By inheriting from Exception class.

OOPs in python

DhruvPatel16120



What is Object-Oriented Programming (OOP)?

Definition: Object-Oriented Programming (OOP) is a programming paradigm that organizes software design around **objects** rather than functions or logic. Objects represent real-world entities and combine **data (attributes)** and **behavior (methods)** into a single unit.
Python supports OOP fully and allows writing both simple and highly complex systems using object-oriented concepts.

Why OOP is Needed
As programs grow larger, procedural code becomes difficult to manage. OOP solves this problem by introducing structure and modularity.

Problems Without OOP

Problem	Description
Code duplication	Same logic repeated
Poor scalability	Hard to extend programs
Low maintainability	Changes break other code
Weak data security	Data exposed everywhere

How OOP Solves These

OOP Feature	Benefit
Objects	Real-world modeling
Classes	Blueprint for reuse
Encapsulation	Data protection
Inheritance	Code reuse
Polymorphism	Flexible behavior
Abstraction	Reduced complexity

bhrugupatel1612

Core Building Blocks of OOP

1. Class

Definition: A **class** is a blueprint or template that defines the structure and behavior of objects.

It describes:

- What data an object will have
- What actions an object can perform

A class itself does not consume memory until an object is created.

2. Object

Definition An **object** is an instance of a class.
It represents a real-world entity with its own identity, state, and behavior.
Each object:

- Has its own copy of attributes
- Can call class methods
- Occupies memory

Class vs Object

Aspect	Class	Object
Nature	Blueprint	Real instance
Memory	No	Yes
Existence	Logical	Physical
Example	Car	my_car

OOP Design Philosophy

OOP is based on the idea that:
"Software should model real-world entities and interactions."

Example mapping:

Real World	OOP Concept
Car	Class
Specific car	Object
Speed, color	Attributes
Drive, brake	Methods

The Four Pillars of OOP (Core Principles)

1. Encapsulation

Definition: Encapsulation is the mechanism of bundling data and methods together and restricting direct access to internal data.
In simple words: Hide internal details and expose only what is necessary.

Dhruvpatel16120

Why Encapsulation Matters

Benefit	Explanation
Data protection	Prevents accidental modification
Controlled access	Only allowed operations
Maintainability	Internal changes don't affect users
Cleaner code	Clear interface

Levels of Access (Conceptual)

Access Level	Meaning
Public	Accessible everywhere
Protected	Intended for subclasses
Private	Internal use only

Python enforces encapsulation by **convention**, not strict rules.

2. Abstraction

Definition: Abstraction means hiding complex implementation details and showing only essential features to the user.

Focus on **what** an object does, not **how** it does it.

Real-World Analogy

Example	Explanation
Car pedals	Driver doesn't know engine logic
Mobile phone	User taps icons, not circuits

Purpose of Abstraction

Purpose	Result
Reduce complexity	Easier understanding
Improve focus	Only relevant details
Increase flexibility	Implementation can change
Support scalability	Easier system growth

Abstraction in Python

Python supports abstraction through:

- Abstract classes

DhruvPatel16120

- Abstract methods
- Interfaces (conceptually)

3. Inheritance

Definition: Inheritance allows a class to acquire properties and behaviors of another class.
Child class reuses and extends parent class functionality.

Why Inheritance is Important

Advantage	Description
Code reuse	Avoids duplication
Logical hierarchy	Parent–child relationship
Faster development	Build on existing logic
Easy maintenance	Changes propagate

Types of Inheritance (Conceptual)

Type	Description
Single	One parent, one child
Multilevel	Parent → child → grandchild
Multiple	Child inherits from multiple parents
Hierarchical	One parent, multiple children
Hybrid	Combination of types

Python supports all these forms.

4. Polymorphism

Definition: Polymorphism means “many forms”.

The same method name can behave differently depending on the object.

Why Polymorphism is Powerful

Benefit	Explanation
Flexibility	Same interface, different behavior
Extensibility	New behavior without changing old code
Clean design	Reduces conditionals
Dynamic execution	Runtime behavior resolution

Types of Polymorphism

Dhruvpatel16120

Type	Description
Compile-time	Method overloading (conceptual)
Runtime	Method overriding

Python mainly uses **runtime polymorphism**.

Additional OOP Concepts (Often Ignored but Important)

Association
Relationship where objects are connected but independent.
Example:

- Teacher ↔ Student

Aggregation
A "has-a" relationship where child can exist independently.
Example:

- Department has Teachers

Composition
Strong "has-a" relationship where child cannot exist alone.
Example:

- House has Rooms

Comparison

Relationship	Dependency
Association	Weak
Aggregation	Medium
Composition	Strong

OOP vs Procedural Programming

Feature	Procedural	OOP
Focus	Functions	Objects
Data security	Weak	Strong
Reusability	Low	High
Scalability	Poor	Excellent
Real-world modeling	Difficult	Natural

Why Python is Strong for OOP

- Simple syntax
- Dynamic typing
- Supports multiple paradigms
- Strong standard library
- Rapid development

Python allows mixing:

- Procedural
- Object-Oriented

DhruvPatel16120

- Functional styles

Common OOP Design Goals

Goal	Meaning
Modularity	Independent components
Reusability	Use code again
Extensibility	Add features easily
Maintainability	Easy updates
Reliability	Fewer bugs

Summary of OOP Theory

- OOP organizes code using **classes and objects**
- Core principles:
  - Encapsulation
  - Abstraction
  - Inheritance
  - Polymorphism
- Promotes clean, scalable, and maintainable software
- Models real-world systems naturally
- Essential for large-scale and professional software development

OOP Implementation in Python (Beginner → Advanced)

1. Creating Classes and Objects (Foundation)

Class Syntax: A class is created using the class keyword. It acts as a blueprint for objects.

Key Components of a Class

Component	Purpose
Class name	Defines blueprint name
Attributes	Store object data
Methods	Define behavior
Constructor	Initializes object

The Constructor (__init__ Method)

Definition: The constructor initializes object attributes when an new object is created.

Key Concepts

- Automatically called during object creation
- Uses self to refer to instance variables
- Helps assign initial values

DrIVPatel16120

Instance Variables vs Class Variables

Instance Variables: Belong to individual objects.

Class Variables: Shared among all objects of the class.

Comparison

Feature	Instance Variable	Class Variable
Ownership	Object-specific	Shared by class
Memory	Separate copy	Single shared copy
Access	Through object	Through class/object

Types of Methods in Python Classes

1. Instance Methods
Operate on object data and use self.

2. Class Methods
Operate on class-level data using cls.

3. Static Methods
Independent utility methods inside class namespace.

Method Comparison

Method Type	Access	Purpose
Instance	Object	Modify instance data
Class	Class	Modify class state
Static	None	Utility logic

Encapsulation in Implementation
Encapsulation is implemented using naming conventions.

Access Levels in Python

Modifier Style	Meaning
Public	Accessible everywhere
Protected (_)	Internal use by convention
Private (__)	Name-mangled protection

Encapsulation also involves getter and setter methods.

Inheritance Implementation

DhruvPatel16120

Definition: Inheritance allows a child class to reuse parent functionality.

Key Concepts

Concept	Meaning
Parent class	Base class
Child class	Derived class
Method overriding	Redefining parent method
super()	Access parent methods

Types of Inheritance in Python

Type	Description
Single	One parent
Multiple	Multiple parents
Multilevel	Chain inheritance
Hierarchical	One parent, many children
Hybrid	Mixed structure

Python supports all inheritance types.

Polymorphism Implementation
Method Overriding:
Child class provides its own version of parent method.

Duck Typing:
Objects are used based on behavior, not type.
"If it behaves like a duck, it is a duck."

Operator Overloading
Python allows redefining operators using special methods.

Abstraction in Python

Abstraction is implemented using abstract base classes.
Key Elements

Element	Purpose
Abstract class	Cannot be instantiated
Abstract method	Must be implemented
Interface behavior	Enforces structure

Python uses abstract base class modules for this.

DhruvPatel16120

Constructors and Destructors

Constructor: Initializes object state.
Destructor: Called when object is destroyed.

Method	Purpose
Constructor	Setup object
Destructor	Cleanup resources

Special (Magic) Methods

Magic methods define object behavior.
Common Magic Methods

Method	Function
__str__	String representation
__repr__	Developer representation
__len__	Length behavior
__eq__	Equality comparison
__add__	Operator overloading

These enable custom object behavior.

Composition and Aggregation in Practice

Composition
Object owns another object strongly.
Aggregation
Objects are loosely connected.

Relationship	Dependency Strength
Composition	Strong
Aggregation	Moderate

Method Resolution Order (MRO)
Definition: MRO determines the order in which Python searches parent classes.
Python uses the **C3 linearization algorithm**.
Importance

- Avoids ambiguity in multiple inheritance
- Ensures consistent behavior

OOP Design Patterns (Intro Level)

Pattern	Purpose

DhruvPatel16120

Singleton	One object instance
Factory	Controlled object creation
Observer	Event-driven updates
Strategy	Interchangeable algorithms

Best Practices in Python OOP

Practice	Reason
Keep classes small	Easier maintenance
Favor composition over inheritance	Flexibility
Use meaningful names	Readability
Avoid deep inheritance	Simplicity
Follow PEP standards	Consistency

Real-World OOP Application Areas

Domain	Usage
Software engineering	System architecture
Game development	Entity modeling
Data science	Model abstraction
Web frameworks	MVC architecture

Summary

OOP implementation in Python involves:

- Creating classes and objects
- Using constructors and variables
- Implementing encapsulation, inheritance, polymorphism, and abstraction
- Leveraging magic methods and composition
- Applying advanced techniques like MRO and design patterns

These concepts allow building scalable, reusable, and maintainable applications from beginner to advanced levels.

Some Useful Libraries in Python for Data Science

No.	Library	Use Case
1.	`math`	Mathematical operations
2.	`statistics`	Mean, median, mode

DhruvPatel1612O

3.	random	Random numbers, sampling
4.	itertools	Efficient looping tools
5.	functools	Functional programming helpers
6.	collections	Optimized data structures
7.	operator	Fast functional operators
8.	heapq	Priority queues
9.	bisect	Binary search operations
10.	json	JSON parsing & serialization
11.	urllib	URL handling & HTTP requests
12.	http	HTTP protocols
13.	os	File & directory operations
14.	sys	System-specific parameters
15.	pathlib	Modern file paths
16.	shutil	File operations (copy, move)
17.	glob	File pattern matching
18.	subprocess	Run system commands
19.	platform	OS information
20.	time	Time-related functions
21.	datetime	Date & time handling
22.	csv	CSV file handling
23.	pickle	Object serialization
24.	threading	Multithreading
25.	multiprocessing	Parallel processing
26.	asyncio	Async programming
27.	concurrent.futures	Thread & process pools

Note: you get some demos in library folder.

Match–Case in Python

1. Introduction

Match–case is a control flow feature introduced in **Python 3.10** called **Structural Pattern Matching**.
It allows a program to compare a value against multiple patterns and execute the block that matches.
It is more powerful than a traditional switch statement because it can match:

- Values
- Sequences (lists/tuples)
- Dictionaries
- Object structures
- Conditions

2. Basic syntax

```
match subject:
    case pattern1:
        # action
    case pattern2:
        # action
    case _:
        # default action
```

Key terms

- **subject** → value being checked
- **pattern** → condition to match
- **case** → branch of execution
- **_** → wildcard (default case)

3. How match–case works

1. Python evaluates the **subject**
2. It checks each case **top to bottom**
3. First matching pattern runs
4. Remaining cases are skipped

There is **no fall-through** like C/C++ switch.

4. Types of patterns

4.1 Literal pattern

Matches exact values.

```
x = 1
match x:
  case 1:
    print("One")
```

Works with:

- integers
- strings
- booleans
- None

4.2 Wildcard pattern (_)

Matches anything.

```
match x:
  case _:
    print("Default")
```

Used as a fallback case.

4.3 Capture pattern (variable binding)

DhruvPatel16120

Stores the matched value in a variable.

```
match x:
    case n:
        print("Value is", n)
```

Here n captures any value.

Important rule: A capture pattern **always matches** unless restricted by a guard.

4.4 OR pattern (|)

Matches multiple possibilities.

```
match x:
    case 1 | 2 | 3:
        print("Small number")
```

Equivalent to logical OR.

4.5 Sequence pattern (lists/tuples)

Matches structure and length.

```
data = [1, 2]

match data:
    case [a, b]:
        print(a, b)
```

Rules:

- Length must match
- Order matters

DhruvPatel16120

Using star pattern:

```
match data:
    case [first, *rest]:
        print(first, rest)
```

4.6 Mapping pattern (dictionaries)
Matches dictionary keys.

```
person = {"name": "A", "age": 20}

match person:
    case {"name": n, "age": a}:
        print(n, a)
```

Extra keys are ignored unless specified.

4.7 Class pattern (object matching)
Matches object attributes.

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

p = Point(1, 2)
match p:
    case Point(x=1, y=2):
        print("Matched")
```

Used in object-oriented design.

4.8 Guard pattern (conditional matching)
Adds an extra condition using if.

```
match x:
    case n if n > 10:
        print("Greater than 10")
```

Pattern matches only if condition is true.

4.9 Nested patterns
Patterns can be combined.

```
data = [1, [2, 3]]
match data:
    case [a, [b, c]]:
        print(a, b, c)
```

4.10 AS pattern (aliasing)

Captures the whole matched value.

DhruvPatel16120

```
match x:
   case 1 | 2 as num:
      print("Matched", num)
```

5. Variable scope rules

Variables created in a case:

- Exist after the match block
- Only created if the case runs

Example:

```
match 5:
   case n:
      pass

print(n)  # n exists
```

6. Irrefutable patterns

Patterns that always match:

- _
- capture variables

These must be placed **last**.

Incorrect:

```
match x:
   case n:   # always matches
      ...
   case 1:   # unreachable
```

7. Differences: match–case vs if–elif

Feature	match–case	if–elif
Pattern matching	Yes	No
Structure matching	Yes	No
Readability	Better for complex patterns	Better for simple logic
Fall-through	No	No
Introduced	Python 3.10	Original Python

8. Advantages

- Cleaner syntax for complex branching
- Supports structural matching
- Reduces nested if statements
- Good for parsing structured data

DhruvPatel16120

- Useful in interpreters and compilers

9. Limitations

- Only available in Python 3.10+
- Can reduce readability if overused
- Slightly slower than simple if statements

10. Best practices

- Use for structured data matching
- Keep cases simple and readable
- Place wildcard _ at the end
- Avoid overly complex nested patterns
- Use guards for extra conditions

11. Practical example

```
def describe(value):
    match value:
        case 0:
            return "Zero"
        case int() as n if n > 0:
            return "Positive integer"
        case str() as s:
            return f"String: {s}"
        case _:
            return "Unknown"
```

Match–case is a structural pattern matching feature in Python 3.10 that allows comparing a value against multiple patterns and executing the matching case block.

1. Docstrings in Python

Definition: A **docstring** (documentation string) is a string literal used to explain what a module, function, class, or method does. It is written inside triple quotes.
It helps developers understand the purpose and usage of code.

Syntax of docstring

```
def add(a, b):
    """Return the sum of a and b."""
    return a + b
```

The docstring appears immediately after the definition.

Types of docstrings

1. Module docstring

Describes the whole file.

```
"""This module provides math utility functions."""
```

Placed at the top of the file.

2. Function docstring

DhruvPatel16120

Explains function behavior.

```
def multiply(a, b):
    """Multiply two numbers and return the result."""
```

3. Class docstring

Describes a class.

```
class Student:
    """Represents a student with name and age."""
```

4. Method docstring

Explains class methods.

Accessing docstrings

Using __doc__:

```
print(add.__doc__)
```

Or using help:

```
help(add)
```

Multi-line docstrings

```
def divide(a, b):
    """
    Divide a by b.

    Parameters:
        a (int): numerator
        b (int): denominator

    Returns:
        float: result of division
    """
```

Importance of docstrings

- Improves readability
- Helps in documentation generation
- Supports maintainability
- Useful for teams and large projects

Best practices

- Use clear and simple language
- Start with a short summary
- Explain parameters and return values
- Follow PEP 257 conventions

2. The Zen of Python

Definition: The **Zen of Python** is a collection of guiding principles for writing good Python code. It was written by Tim Peters.

You can view it using:

```
import this
```

DhruvPatel16120

The Zen of Python (key principles explained)

1. Beautiful is better than ugly
Code should be clean and readable.

2. Explicit is better than implicit
Avoid hidden behavior; make intentions clear.

3. Simple is better than complex
Prefer easy solutions.

4. Complex is better than complicated
Some complexity is acceptable if structured.

5. Readability counts
Code should be understandable by humans.

6. There should be one obvious way to do it
Avoid multiple confusing approaches.

7. Errors should never pass silently
Handle exceptions properly.

8. Practicality beats purity
Focus on useful solutions.

9. If implementation is hard to explain, it is a bad idea
Keep logic clear.

Importance

- Encourages clean coding
- Improves collaboration
- Promotes maintainable software

3. PEP 8 (Python Enhancement Proposal 8)
Definition: PEP 8 is the official style guide for Python code. It defines formatting rules to make code consistent and readable.

Key PEP 8 rules

1. Indentation
Use **4 spaces** per indentation level.

```
if True:
    print("Hello")
```

DhruvPatel16120

2. Line length
Limit lines to **79 characters**.

3. Blank lines

- Two blank lines before top-level functions/classes
- One blank line inside classes

4. Naming conventions

Item	Style	Example
Variables	lowercase_with_underscores	user_name
Functions	lowercase_with_underscores	calculate_sum
Classes	CapitalizedWords	StudentData
Constants	UPPERCASE	MAX_SIZE

5. Imports

Imports should be at the top of the file.

```
import os
import sys
```

Order:

1. Standard library
2. Third-party libraries
3. Local modules

6. Whitespace rules
Correct:
x = a + b
Incorrect:
x=a+b
No extra spaces inside parentheses:
func(a, b)

7. Comments
Use clear comments.
# Calculate total price
total = price * quantity

8. Maximum nesting
Avoid deep nesting; keep logic simple.

Benefits of PEP 8

- Improves readability

DhruvPatel16120

- Creates uniform coding style
- Helps teamwork
- Makes maintenance easier

Tools that enforce PEP 8

- flake8
- pylint
- black (auto formatter)

If–Else in One Line in Python (Ternary Operator)

Definition : Python allows writing a simple **if–else statement in one line** using the **ternary conditional expression**.
It is used to assign a value or return a result based on a condition.

Basic syntax

```
value_if_true if condition else value_if_false
```

Structure:

- First part → result if condition is true
- Middle → condition
- Last part → result if condition is false

Simple example

```
x = 10
result = "Positive" if x > 0 else "Negative"
print(result)
```

Output:
Positive

Example with numbers

```
a = 5
b = 8
max_value = a if a > b else b
```

Finds the larger number.

Using in print statement

```
n = 4
print("Even" if n % 2 == 0 else "Odd")
```

Nested one-line if–else

Multiple conditions can be written:

```
x = 0
result = "Positive" if x > 0 else "Negative" if x < 0 else "Zero"
```

Advantages

- Saves lines of code
- Makes simple conditions concise
- Useful in assignments and returns

DhruvPatel16120

Limitations

- Reduces readability if too complex
- Not suitable for long logic
- Avoid deep nesting

Comparison: normal vs one-line

Normal if–else

```
if x > 0:
   result = "Positive"
else:
   result = "Negative"
```

One-line version

```
result = "Positive" if x > 0 else "Negative"
```

Best practices

- Use only for simple conditions
- Avoid long nested expressions
- Prefer readability over brevity

DhruvPatel16120